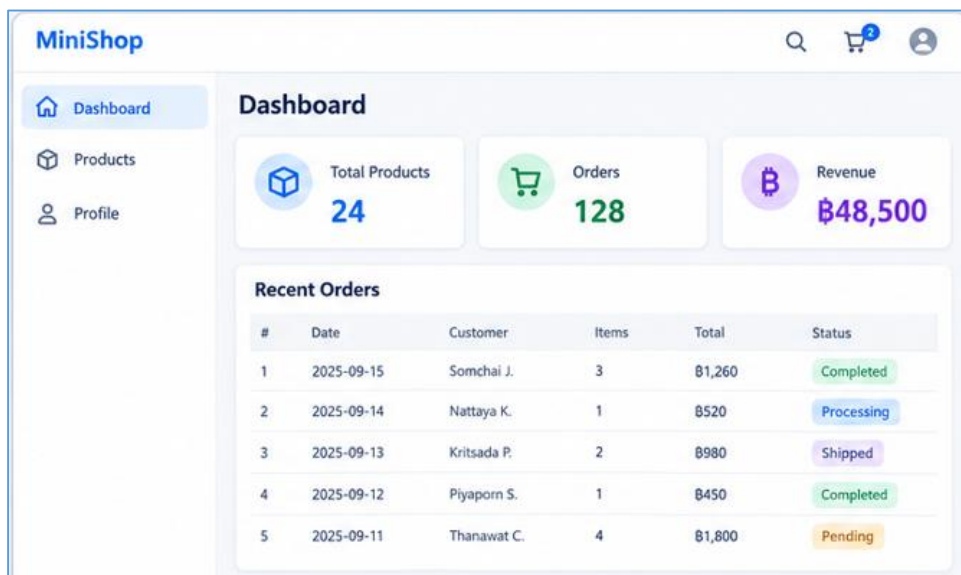
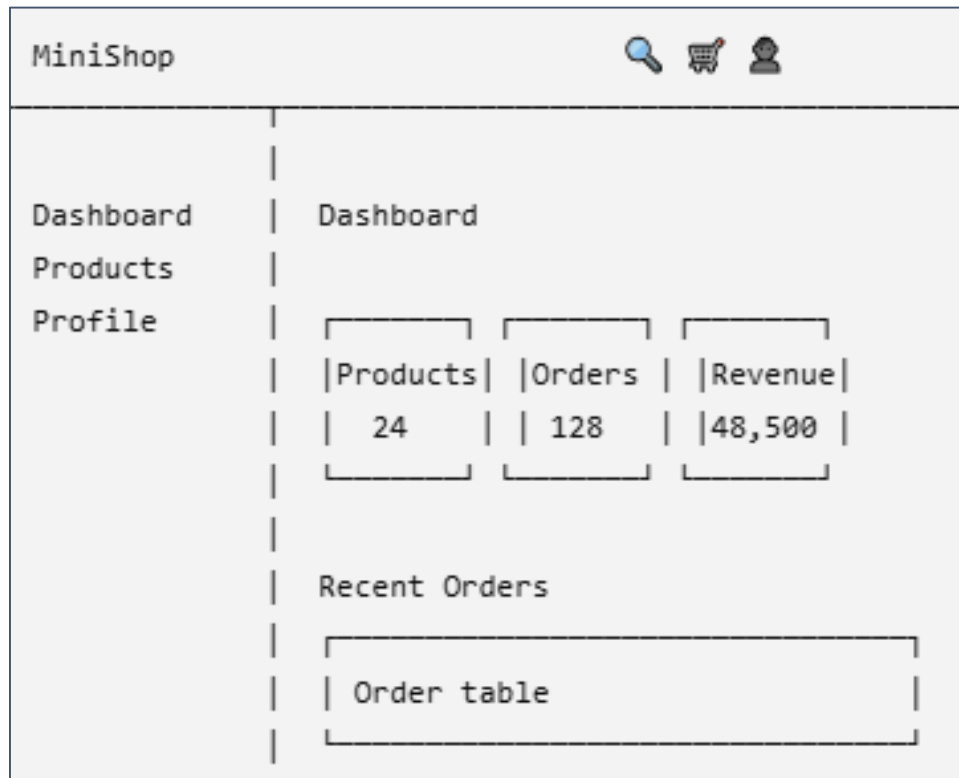
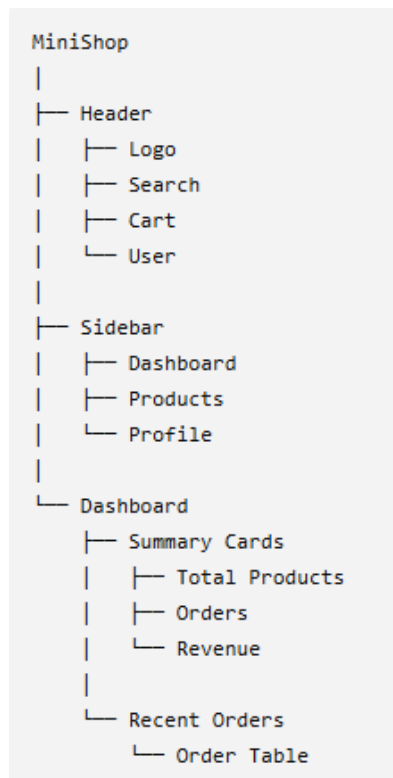


ใบงาน Basic Tailwind and React

ให้นักศึกษาสร้างระบบร้านค้าออนไลน์ด้วย Tailwind and React ตามขั้นตอนในใบงาน โดยใบงานนี้
เริ่มต้นเราจะใช้ **Vite + Vanilla + JavaScript + Tailwind** จากนั้นค่อยปรับไปหา React เพื่อที่นักศึกษาจะได้
เข้าใจและเห็นความแตกต่างได้อย่างชัดเจน เมื่อทำเสร็จแล้วจะได้ผลลัพธ์ประมาณนี้



โครงสร้างเว็บ



Part 1 การติดตั้ง Node.js

ขั้นที่ 1: ดาวน์โหลด Node.js

ให้เข้าเว็บไซต์ทางการของ Node.js คือ https://nodejs.org/en/download?utm_source=chatgpt.com
จากนั้นเลื่อนลงมาด้านล่าง เลือก **LTS** ไม่ต้องเลือก Current

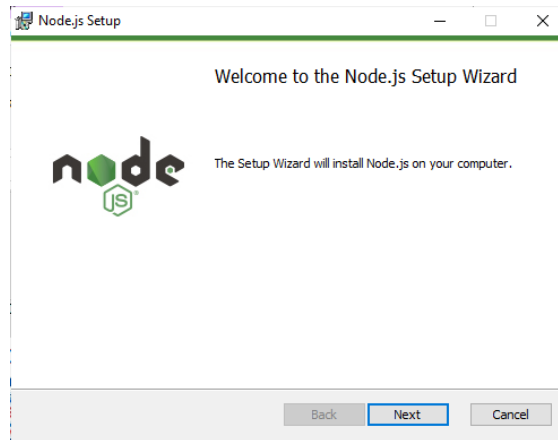


ขั้นที่ 2: ติดตั้ง Node.js

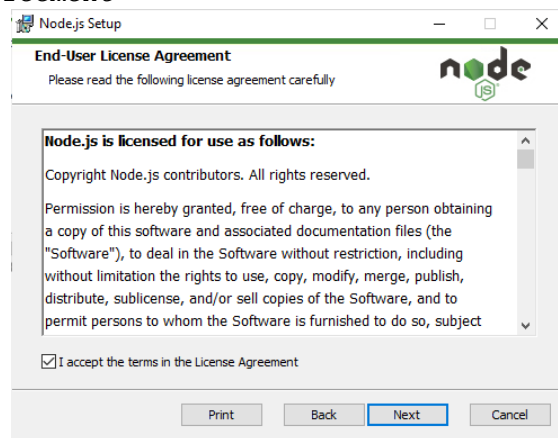
ให้ดาวน์โหลดไฟล์ .msi แล้ว ให้ดับเบิลคลิก ให้เลือกไฟล์ .msi เวอร์ชันอาจจะไม่ตรงกันก็ได้

Windows 64-bit Installer: <https://nodejs.org/dist/v24.21.0/node-v24.21.0-x64.msi>

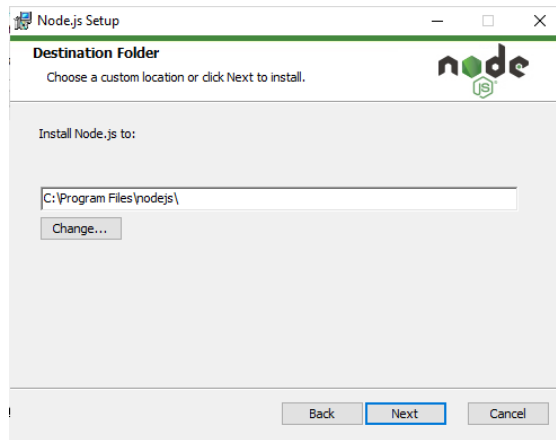
จากนั้นคลิก Next



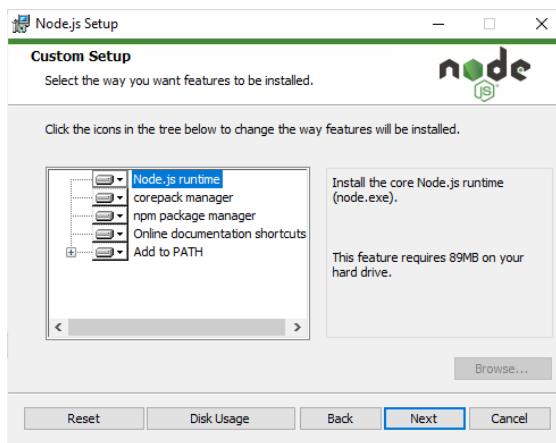
Accept the License Agreement



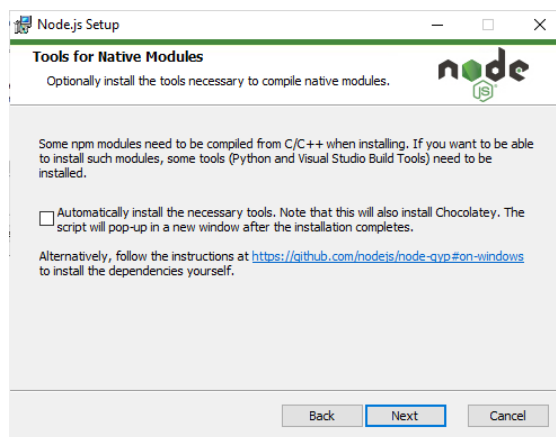
Next



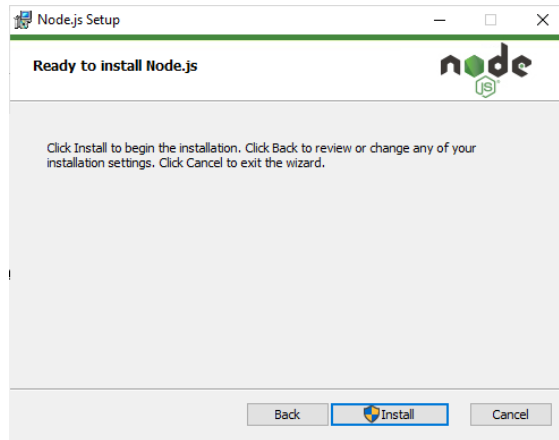
Next



Next



Install



ข้อควรระวัง

ระหว่างติดตั้ง **ไม่ต้องเปลี่ยนค่าอะไร** โดยเฉพาะส่วนที่เกี่ยวกับ Add to PATH ให้ใช้ค่า Default เพราะเราต้องการให้ Windows สามารถเรียก node, npm, npx จาก Terminal ได้

ขั้นที่ 3: ปิด VS Code

หลังติดตั้งเสร็จ **ปิด VS Code ทั้งหมด** แล้วเปิด VS Code ใหม่ เพื่อ VS Code ได้รับค่า PATH ใหม่

ขั้นที่ 4: เปิด Terminal

ใน VS Code ให้เปิด Terminal ใหม่ โดยการเลือกเมนู View -> Terminal จากนั้นทำการตรวจสอบเวอร์ชันของ node, npm และ npx ด้วยคำสั่งต่อไปนี้

```
C:\web_101\minishop>node -v          กดปุ่ม Enter  
v24.x.x
```

```
C:\web_101\minishop>npm -v           กดปุ่ม Enter  
11.x.x
```

```
C:\web_101\minishop>npx -v          กดปุ่ม Enter  
11.x.x
```

ถ้าทุกอย่างถูกต้องจะแสดงเวอร์ชันของแต่ละตัวออกมาดังภาพ

```
Problems Output Debug Console Terminal Ports SQL
(c) Microsoft Corporation. All rights reserved.

C:\web_101\minishop>node -v
v24.21.0

C:\web_101\minishop>npm -v
11.19.0

C:\web_101\minishop>npx -v
11.19.0

C:\web_101\minishop>
```

Part 2 สร้าง Vite Project

ก่อนอื่นเราจะทำการสร้าง Project ด้วย Vite ซึ่ง Vite (อ่านว่า “วีต”) คือ เครื่องมือสำหรับสร้างและพัฒนาเว็บ Application สมัยใหม่ โดยเฉพาะ Project ที่ใช้ React, Vue, Svelte หรือ JavaScript โดย Vite จะช่วยสร้างโครงสร้าง Project และเปิด Web Server สำหรับพัฒนา web application ตามปกติเมื่อเราต้องการสร้าง Project เราต้องสร้างสิ่งเหล่านี้เองทั้งหมด

สร้าง Folder

สร้าง HTML

สร้าง JavaScript

สร้าง Configuration

ติดตั้ง Package

ตั้งค่า Development Server

...

จะเห็นว่าเมื่อสร้าง Vite Project เราจะได้สภาพแวดล้อมสำหรับพัฒนา Web application ทันที ที่สำคัญที่ควรจำคือ Vite ไม่ใช่ React ทำหน้าที่คนละอย่างกัน ตารางด้านล่างเป็นการเปรียบเทียบให้เห็นความแตกต่างชัดเจนยิ่งขึ้น

	หน้าที่
HTML	โครงสร้างหน้าเว็บ
CSS	ตกแต่ง

JavaScript	ทำให้เว็บทำงาน
Tailwind CSS	ช่วยเขียน CSS ด้วย Utility Classes
React	สร้าง UI ด้วย Component
Vite	สร้าง/รัน/จัดการ Development Environment

Step 1 สร้างโฟลเดอร์สำหรับโปรเจก

ในที่นี้ให้สร้างโฟลเดอร์ดังนี้ C:\web_101\minishop

1.1) จากนั้นใน VS Code ให้เลือก Open Folder.. -> minishop

1.2) เปิด Terminal แล้วพิมพ์คำสั่งดังนี้

npm create vite@latest

จากนั้นใส่ชื่อ Project ในที่นี้ให้ใส่ minishop-dashbaord

Project name:

minishop-dashboard

ส่วนของ Select framework ให้เลือก Vanilla ซึ่งเป็นการใช้ JS สำหรับการพัฒนาเว็บ

Select a framework:

Vanilla

Select a variant:

JavaScript

Install with npm and start now?

Yes

***แต่ถ้าต้องการลดจำนวนขั้นตอน เราสามารถใช้คำสั่งนี้ได้

```
npm create vite@latest minishop-dashboard -- --template vanilla
```

หรือ

```
npx create-vite minishop-dashboard
```

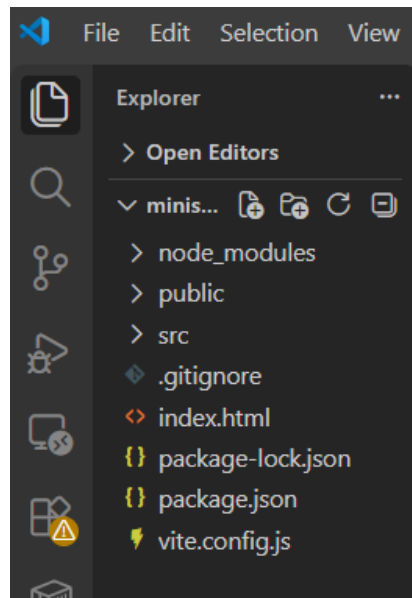
เราจะได้ Project ที่ชื่อ minishop-dashboard

1.3) ติดตั้ง dependencies

```
npm install
```

 กด Enter

ใน VS Code ควรจะขึ้นโครงสร้างโปรเจกต์ดังรูป แต่อาจจะมีไฟล์ vite.config.js ซึ่งไม่เป็นไร เดี่ยวค่อยสร้างใหม่เองได้



Step 2 รัน Vite Server

2.1) เปิด Terminal ใหม่ แล้วพิมพ์คำสั่ง **npm run dev** จะได้หน้าต่างประมาณนี้

```
Problems Output Debug Console Terminal Ports SQL His
Microsoft Windows [Version 10.0.19045.6456]
(c) Microsoft Corporation. All rights reserved.

C:\web_101\minishop\minishop-dashboard>npm run dev

> minishop-dashboard@0.0.0 dev
> vite

15:03:06 [vite] (client) Re-optimizing dependencies b

VITE v8.3.0 ready in 499 ms

→ Local: http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

Step 3 ติดตั้ง Tailwind CSS

3.1) เปิด Terminal ใหม่ขึ้นมาอีกอัน จากนั้นพิมพ์คำสั่งเพื่อติดตั้ง Tailwind ดังนี้

npm install tailwindcss @tailwindcss/vite

กด Enter

3.2) รอสักครู่ สุดท้ายจะแสดง **found 0 vulnerabilities** แสดงว่า ปกติ

สรุป ตอนนี้เราจะมี Terminal 2 หน้าต่าง คือหน้าต่างสำหรับรัน Vite server กับหน้าต่างสำหรับพิมพ์คำสั่งติดตั้ง package และคำสั่งอื่นๆ

3.3) ตรวจสอบว่า Installation สำเร็จหรือไม่

เปิดไฟล์

package.json

ใน VS Code มองหาส่วน

"dependencies"

หรือ

"devDependencies"

ควรพบ Package ที่เกี่ยวข้องกับ Tailwind เช่น

"tailwindcss": "...",

"@tailwindcss/vite": "..."

Version ไม่จำเป็นต้องตรงกับตัวอย่างในใบงานเพราะ package อาจเปลี่ยน version ในอนาคต

3.4) แก้ไขไฟล์ vite.config.js

กรณีที่มีไฟล์อยู่แล้ว ให้ลบโค้ดเดิมออกให้หมด จากนั้นแทนที่ด้วยโค้ดใหม่ด้านล่าง กดบันทึก
กรณีที่ยังไม่มีไฟล์นี้ ให้ทำการสร้างไฟล์นี้ขึ้นมาใหม่แล้วเพิ่มโค้ดด้านล่างลงไป กดบันทึก

```
import { defineConfig } from 'vite'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins:[
    tailwindcss()
  ]
})
```

อธิบายโค้ดทีละบรรทัด

บรรทัดที่ 1

```
import { defineConfig } from 'vite'
```

นำ defineConfig จาก Vite มาใช้สร้าง Configuration

บรรทัดที่ 2

```
import tailwindcss from '@tailwindcss/vite'
```

นำ Tailwind Vite Plugin เข้ามาใช้

บรรทัดที่ 4

```
export default defineConfig({
```

กำหนด Configuration ของ Vite

บรรทัดที่ 5

```
plugins: [
```

บอกว่า Project นี้จะใช้ Plugin

บรรทัดที่ 6

```
tailwindcss()
```

เปิดใช้งาน Tailwind CSS Plugin

3.5) แก้ไขไฟล์ src/style.css

ก่อนจะทำการแก้ไข ให้นักศึกษาทำการสำรองไฟล์เดิมไว้ก่อน จากนั้นให้ทำการลบคำสั่งทั้งหมดในไฟล์ style.css จากนั้นให้พิมพ์คำสั่ง

```
@import "tailwindcss";
```

ไว้บรรทัดแรก จากนั้นบันทึกไฟล์

3.6) ตรวจสอบว่า CSS ถูก Import หรือไม่

เปิดไฟล์ main.js ที่อยู่ใน src/main.js

จากนั้นลบคำสั่งทั้งหมด

แทนที่ด้วยคำสั่งต่อไปนี้ แล้วกดบันทึก

```
import './style.css'

document.querySelector('#app').innerHTML = `
  <div class="min-h-screen bg-gray-100 p-10">

    <h1 class="text-4xl font-bold text-blue-600">
      MiniShop
    </h1>

    <p class="mt-4 text-gray-600">
      Tailwind CSS is working!
    </p>

    <button class="mt-6 bg-blue-600 text-white px-6 py-3 rounded-
lg hover:bg-blue-700">
      Test Button
    </button>

  </div>
`
```

ดูผลใน Browser ถ้า Tailwind ทำงาน ควรจะได้ผลลัพธ์ดังรูป



โดย

- ☐ MiniShop ตัวใหญ่
- ☐ ตัวอักษรสีน้ำเงิน
- ☐ พื้นหลังสีเทาอ่อน
- ☐ ปุ่มสีน้ำเงิน
- ☐ ปุ่มมีมุมโค้ง
- ☐ เมื่อเอา Mouse ไปวาง ปุ่มเปลี่ยนสี

PART 3 Tailwind Utility Classes

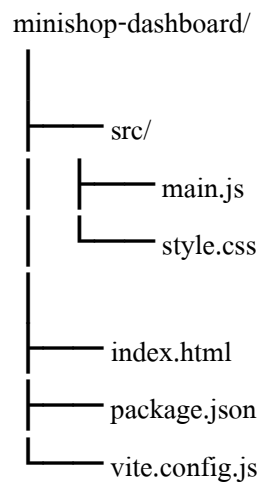
หลังจากทำ Part นี้ นักศึกษาสามารถ

- ☐ ใช้ text-* กำหนดขนาดตัวอักษร
- ☐ ใช้ font-* กำหนดน้ำหนักตัวอักษร
- ☐ ใช้ bg-* กำหนดสีพื้นหลัง
- ☐ ใช้ p-* และ m-* จัดระยะห่าง

- ☐ ใช้ w-* และ h-* กำหนดขนาด
- ☐ ใช้ border-* สร้างเส้นขอบ
- ☐ ใช้ rounded-* ทำมุมโค้ง
- ☐ ใช้ shadow-* สร้างเงา
- ☐ ใช้ hover:* สร้าง Interaction
- ☐ นำ Utility Classes หลายตัวมาประกอบเป็น UI จริง

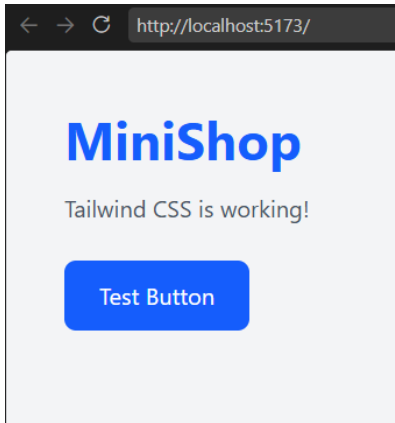
3.1 ก่อนเริ่ม

จาก PART 2 นักศึกษาควรมี



และ Browser ควรจะเปิดได้ที่ <http://localhost:5173/>

โดยเห็น



3.2 Utility Class คืออะไร

Tailwind ให้เราใช้ Class ขนาดเล็ก ๆ เพื่อกำหนด Style

ตัวอย่าง

```
<h1 class="text-4xl font-bold text-blue-600">
```

MiniShop

```
</h1>
```

แทนที่จะเขียน CSS แบบเดิม

```
.title {  
  font-size: 2.25rem;  
  font-weight: 700;  
  color: #2563eb;  
}
```

เราสามารถอ่าน

text-4xl

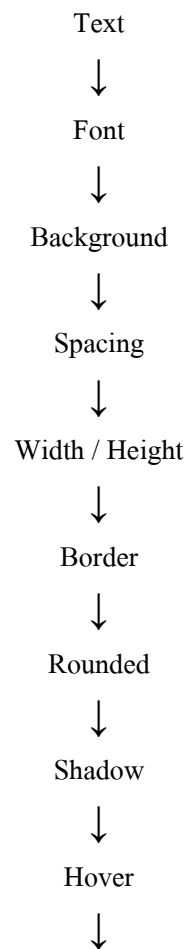
font-bold

text-blue-600

แล้วประกอบ UI ได้ทันที

3.3 Utility ที่เราจะเรียนใน Part นี้

ให้จำ Flow นี้ไว้



ประกอบเป็น MiniShop Header

text-*

3.4 ทดลองกำหนดขนาดตัวอักษร

เปิด

src/main.js

ลบ Code เดิมทั้งหมด แล้วแทนที่ด้วยโค้ดนี้

```
import './style.css'

document.querySelector('#app').innerHTML = `
  <div>
    <p class="text-sm">
      Small Text
    </p>

    <p class="text-base">
      Base Text
    </p>

    <p class="text-xl">
      Extra Large Text
    </p>

    <p class="text-4xl">
      Very Large Text
    </p>
  </div>
`
```

</div>

กด **Ctrl + S** แล้วดู Browser ควรเห็นประมาณ

Small Text

Base Text

Extra Large Text

Very Large Text

โดยขนาดจะเพิ่มขึ้นตามลำดับ text-sm -> text-base -> text-xl -> text-4xl (ลองเปลี่ยนเป็น text-6xl ดูผลลัพธ์)

font-*

3.5 ทดลองนำหน้าทัวอักษร

เพิ่ม

<p class="font-normal">

Normal

</p>

<p class="font-medium">

Medium

</p>

<p class="font-semibold">

Semi Bold

</p>

<p class="font-bold">

Bold

</p>

ผลลัพธ์

Normal

Medium

Semi Bold

Bold

ทดลองผสม Class

```
<h1 class="text-4xl font-bold">  
  MiniShop  
</h1>
```

ตรงนี้ให้สังเกตว่า

text-4xl+font-bold

สามารถใช้ร่วมกันได้

text-* สำหรับสี

3.6 กำหนดสีตัวอักษร

ทดลอง

```
<p class="text-blue-600">  
  Blue Text  
</p>
```

```
<p class="text-green-600">  
  Green Text  
</p>
```

```
<p class="text-red-600">  
  Red Text  
</p>
```

```
<p class="text-gray-600">
  Gray Text
</p>
```

ตัวอย่างการใช้จริง

```
<h1 class="text-3xl font-bold text-blue-600">
  MiniShop
</h1>
```

bg-*

3.7 กำหนดสีพื้นหลัง

ทดลอง

```
<div class="bg-blue-600 text-white p-4">
  Blue Box
</div>
```

ลองเปลี่ยนเป็น

bg-green-600

แล้ว

bg-gray-100

สังเกตความแตกต่าง นักศึกษาจะพบว่า

text-blue-600 หมายถึง สีของตัวอักษร แต่bg-blue-600 หมายถึง สีพื้นหลัง

p-* Padding

3.8 ทดลอง Padding

เขียน

```
<div class="bg-blue-600 text-white p-4">
  MiniShop
</div>
```

ลองเปลี่ยน

p-2
p-4
p-6
p-8

สังเกตว่าเนื้อหากับขอบกล่องห่างกันมากขึ้น

m-* Margin

3.9 ทดลอง Margin

```
<div class="bg-blue-600 text-white p-4">
  Box 1
</div>
```

```
<div class="bg-green-600 text-white p-4 mt-6">
  Box 2
</div>
```

mt-6 หมายถึง

margin-top

ลองเปลี่ยน

mt-2
mt-4
mt-8

w-* และ h-*

3.10 กำหนดความกว้าง

```
<div class="w-64 bg-blue-600 text-white p-4">
  Width 64
</div>
```

ทดลอง

w-32

w-48
w-64
w-full

3.11 กำหนดความสูง

```
<div class="w-64 h-32 bg-blue-600 text-white p-4">  
  MiniShop  
</div>
```

ทดลอง

h-24
h-32
h-48

border-*

3.12 เพิ่มเส้นขอบ

```
<div class="border p-4">  
  Product Card  
</div>
```

ทดลอง

```
<div class="border border-gray-300 p-4">  
  Product Card  
</div>
```

และ

```
<div class="border-2 border-blue-500 p-4">  
  Product Card  
</div>
```

rounded-*

3.13 ทำมุมโค้ง

ทดลอง

```
<div class="bg-white border rounded p-4">  
  Small Rounded  
</div>
```

จากนั้น

```
rounded  
rounded-md  
rounded-lg  
rounded-xl  
rounded-2xl
```

ตัวอย่างที่เราจะใช้กับ MiniShop

```
<div class="bg-white rounded-xl p-6">
```

shadow-*

3.14 เพิ่มเงา

```
<div class="bg-white p-6 shadow-md">  
  Product Card  
</div>
```

ทดลองเปลี่ยนค่า แล้วสังเกตความแตกต่างของเงา

```
shadow-sm  
shadow  
shadow-md  
shadow-lg  
shadow-xl
```

hover:*

ตอนนี้เราจะเริ่มทำให้ UI มี Interaction

3.15 สร้าง Button

```
<button class="bg-blue-600 text-white px-6 py-3 rounded-lg">
  Add to Cart
</button>
```

จากนั้นเพิ่ม

```
hover:bg-blue-700
```

กลายเป็น

```
<button
  class="bg-blue-600 text-white px-6 py-3 rounded-lg hover:bg-blue-700">

  Add to Cart

</button>
```

เมื่อเอา Mouse ไปวางบนปุ่มจะเห็นว่า
ปกติ

Blue 600

↓ hover

Blue 700

รวมทุกอย่างเข้าด้วยกัน

ตอนนี้เราจะสร้าง **MiniShop Header** จริง ให้ลบ Code ใน main.js แล้วใส่

```
import './style.css'
```

```
document.querySelector('#app').innerHTML = `
```

```
<header class="bg-white shadow-md">
```

```
  <div class="max-w-7xl mx-auto px-6 py-4
    flex items-center justify-between">
```

```
    <!-- Logo -->
```

```
  </div>
```

```

        <h1 class="text-2xl font-bold text-blue-600">
            MiniShop
        </h1>

        <p class="text-sm text-gray-500">
            Online Shopping
        </p>
    </div>

    <!-- Search -->

    <div class="flex items-center gap-4">

        <input
            type="text"
            placeholder="Search products..."
            class="border border-gray-300
                rounded-lg
                px-4 py-2
                w-64"
        >

        <button
            class="bg-blue-600
                text-white
                px-4 py-2
                rounded-lg
                hover:bg-blue-700">

Search

        </button>

    </div>

    <!-- Cart -->

    <button
class="bg-gray-100
    px-4 py-2
    rounded-lg
    hover:bg-gray-200">

 Cart

    </button>

```

</div>

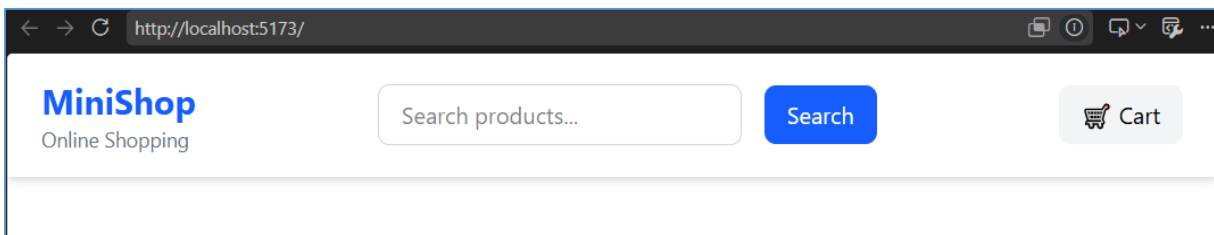
</header>

,

กด

Ctrl + S

☐ ผลลัพธ์ที่ควรได้



และ Header ว่าเราใช้ Utility อะไรบ้าง

ตอนนี้ให้นักศึกษาย้อนกลับไปดู Code

Header

bg-white
shadow-md

หมายถึง พื้นหลังสีขาว + เงา

Container

max-w-7xl
mx-auto
px-6
py-4

หมายถึง จำกัดความกว้าง + จัดให้อยู่ตรงกลาง + padding ซ้ายขวา + padding บนล่าง

Flex

flex
items-center
justify-between

หมายถึง Flexbox + จัดแนวกกลาง + กระจายซ้าย/ขวา

Logo

text-2xl
font-bold
text-blue-600

หมายถึง ขนาดตัวอักษร + ตัวหนา + สีน้ำเงิน

Search

border
rounded-lg
px-4
py-2
w-64

หมายถึง เส้นขอบ + มุมโค้ง + padding + ความกว้าง

Button

bg-blue-600
text-white
rounded-lg
hover:bg-blue-700

หมายถึง พื้นหลังน้ำเงิน + ตัวอักษรขาว + มุมโค้ง + เปลี่ยนสีเมื่อ Hover

PART 4 — Flexbox & Layout

4.1 วัตถุประสงค์

เมื่อจบ Part นี้ นักศึกษาควรจะสามารถ

1. อธิบายหลักการทำงานของ Flexbox ได้
2. ใช้ flex จัด Layout ได้
3. ใช้ items-center จัดตำแหน่งแนวตั้งได้
4. ใช้ justify-between จัดตำแหน่งซ้าย-ขวาได้
5. ใช้ gap-* กำหนดระยะห่างได้
6. ใช้ max-w-* และ mx-auto จำกัดความกว้างและจัดให้อยู่กึ่งกลางได้

4.2 ทำความเข้าใจก่อนเขียน Code

ลองดู HTML ปกติ:

```
<div>
  <h1>MiniShop</h1>
  <button>Cart</button>
</div>
```

Browser จะเรียงองค์ประกอบตามปกติ

```
MiniShop
Cart
```

แต่เราต้องการ:

```
MiniShop                                Cart
```

เราสามารถใช้ Tailwind:

```
<div class="flex justify-between">
```

ได้ทันที

4.3 flex

Concept

flex

ใช้เปิดใช้งาน Flexbox

ตัวอย่าง:

```
<div class="flex">
  <div>Box 1</div>
  <div>Box 2</div>
  <div>Box 3</div>
</div>
```

ผลลัพธ์



ถ้าไม่ใช่ flex:

```
Box 1
Box 2
Box 3
```

4.4 ทดลอง flex

เปิดไฟล์ main.js ที่อยู่ใน src/main.js

เปลี่ยนโค้ดให้เป็น

```
import './style.css'

document.querySelector('#app').innerHTML = `

<div class="p-10">

  <h1 class="text-2xl font-bold mb-6">
    Flexbox Demo
  </h1>

  <div class="flex gap-4">

    <div class="bg-blue-500 text-white p-6 rounded-lg">
```

```

        Box 1
    </div>

    <div class="bg-green-500 text-white p-6 rounded-lg">
        Box 2
    </div>

    <div class="bg-purple-500 text-white p-6 rounded-lg">
        Box 3
    </div>

</div>

</div>

```

สังเกตว่า flex ทำให้ Box ทั้ง 3 อยู่ในแนวนอน ส่วน gap-4 ทำให้เกิดช่องว่างระหว่าง Box

4.5 gap-*

ตัวอย่าง:

```

<div class="flex gap-2">
<div class="flex gap-4">
<div class="flex gap-8">

```

ความแตกต่างคือระยะห่าง:

gap-2 ☐ เล็ก
 gap-4 ☐ ปานกลาง
 gap-8 ☐ ใหญ่

ทดลอง เปลี่ยน gap-4 เป็น gap-8 แล้วสังเกตความแตกต่าง

4.6 items-center

สมมติว่า:

```

<div class="flex">

```

มี Logo และปุ่มที่มีความสูงไม่เท่ากัน เราสามารถใช้ **items-center** เพื่อจัดให้อยู่กึ่งกลางในแนวตั้ง

```
<div class="flex items-center">
```

4.7 justify-between

ตัวอย่าง:

```
<div class="flex justify-between">
```

จะกระจายสมาชิกไปคนละด้าน

MiniShop	Cart
↑	↑
ซ้าย	ขวา

4.8 items-center + justify-between

สอง class นี้มักใช้ร่วมกันมาก:

```
<div class="flex items-center justify-between">
```

ความหมาย

flex
↓
จัดเป็น Flexbox

items-center
↓
จัดกึ่งกลางแนวตั้ง

justify-between
↓
แยกซ้าย-ขวา

4.9 max-w-7xl

ต่อไปเราจะเริ่มทำ Layout จริงของ MiniShop

```
<div class="max-w-7xl">
```

max-w-7xl หมายถึง จำกัดความกว้างสูงสุดของ Container ช่วยไม่ให้เว็บไซต์กว้างเต็มจอจนเกินไปบนจอขนาดใหญ่

4.10 mx-auto

```
<div class="mx-auto">
```

หมายถึง:

```
margin-left:auto  
margin-right:auto
```

ผลคือ Container จะอยู่ตรงกลาง ดังนั้นเรามักเห็น

```
<div class="max-w-7xl mx-auto">
```

ใช้คู่กัน

max-w-7xl



จำกัดความกว้าง

mx-auto



จัดให้อยู่กึ่งกลาง

4.11 สร้าง MiniShop Header

ตอนนี้ให้สร้าง Header จริง

```
import './style.css'

document.querySelector('#app').innerHTML = `

<header class="bg-white shadow-md">

  <div class="max-w-7xl mx-auto px-6 py-4
    flex items-center justify-between">

    <!-- Logo -->
    <div>
      <h1 class="text-2xl font-bold text-blue-600">
        MiniShop
      </h1>

      <p class="text-sm text-gray-500">
        Online Shopping
      </p>
    </div>

    <!-- Search -->
    <div class="flex items-center gap-3">

      <input
        type="text"
        placeholder="Search products..."
        class="border border-gray-300
          rounded-lg
          px-4 py-2
          w-64"
        >

      <button
        class="bg-blue-600
          text-white
          px-4 py-2
          rounded-lg
          hover:bg-blue-700">

        Search

      </button>

    </div>

  </div>

</header>`
```

```
<!-- Cart -->
<button
  class="bg-gray-100
    px-4 py-2
    rounded-lg
    hover:bg-gray-200">
```

Cart

</button>

</div>

</header>

—

4.12 วิเคราะห์ Code

ส่วนนี้

```
<divclass="max-w-7xl mx-auto px-6 py-4
      flex items-center justify-between">
```

มีหลาย Utility

Class	ความหมาย
max-w-7xl	จำกัดความกว้าง
mx-auto	จัดให้อยู่กึ่งกลาง
px-6	Padding ซ้าย-ขวา
py-4	Padding บน-ล่าง
flex	เปิด Flexbox
items-center	จัดกึ่งกลางแนวตั้ง

Class ความหมาย

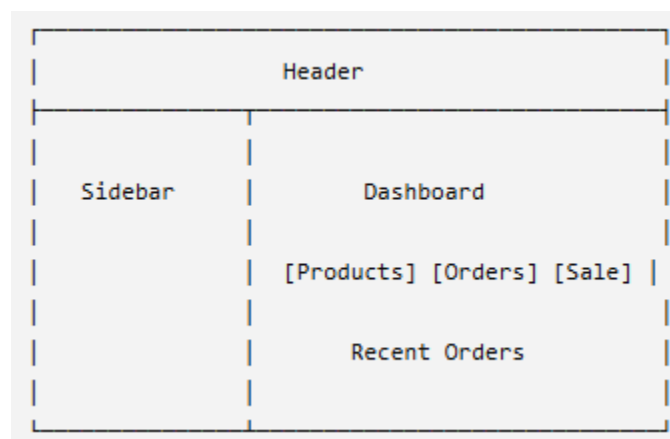
justify-between กระจายซ้าย-ขวา

PART 5 — MiniShop Dashboard

ตอนนี้เราจะนำความรู้จาก Part 4 มาสร้าง Dashboard จริง

5.1 โครงสร้าง Dashboard

UI ที่เราต้องการ



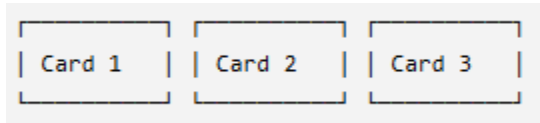
ดังนั้นเราจะใช้ Flex + Grid

5.2 grid

ใน Tailwind มี grid สำหรับสร้าง Layout แบบ Grid ตัวอย่าง

```
<div class="grid grid-cols-3 gap-6">
```

หมายถึง สร้าง Grid -> 3 columns -> มีระยะห่าง 6 ผลลัพธ์จะได้ดังภาพ

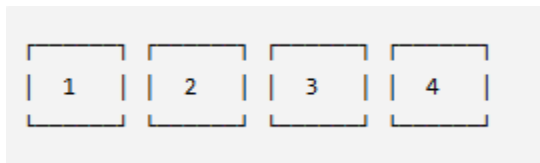


5.3 grid-cols-4

ถ้าต้องการ 4 columns

```
<div class="grid grid-cols-4 gap-6">
```

ผลลัพธ์



5.4 สร้าง Dashboard Layout

แทนที่ main.js ด้วยโค้ดต่อไปนี้

```
import './style.css'

document.querySelector('#app').innerHTML = `

<div class="min-h-screen bg-gray-100">

  <!-- ===== HEADER ===== -->

  <header class="bg-white shadow-md">

    <div class="max-w-7xl mx-auto px-6 py-4
      flex items-center justify-between">

      <div>
```

```

        <h1 class="text-2xl font-bold text-blue-600">
            MiniShop
        </h1>

        <p class="text-sm text-gray-500">
            Admin Dashboard
        </p>
    </div>

    <div class="flex items-center gap-4">

        <button
            class="bg-gray-100 px-4 py-2 rounded-lg
                hover:bg-gray-200">

```

□ Notification

```

        </button>

        <button
            class="bg-blue-600 text-white
                px-4 py-2 rounded-lg
                hover:bg-blue-700">

            Admin
        </button>

    </div>

</div>

</header>

```

<!--=====MAIN=====-->

```

<main class="max-w-7xl mx-auto px-6 py-8">

    <h2 class="text-3xl font-bold text-gray-800">
        Dashboard
    </h2>

    <p class="mt-2 text-gray-500">
        Welcome back to MiniShop
    </p>

```

<!--=====STAT CARDS=====-->

```

<div class="grid grid-cols-4 gap-6 mt-8">

```

```

<!-- Card 1 -->
<div class="bg-white p-6 rounded-xl shadow">

  <p class="text-gray-500">
    Total Products
  <p>

  <h3 class="text-3xl font-bold mt-2">
    24
  </h3>

</div>

<!-- Card 2 -->
<div class="bg-white p-6 rounded-xl shadow">

  <p class="text-gray-500">
    Orders
  <p>

  <h3 class="text-3xl font-bold mt-2">
    128
  </h3>

</div>

<!-- Card 3 -->
<div class="bg-white p-6 rounded-xl shadow">

  <p class="text-gray-500">
    Revenue
  <p>

  <h3 class="text-3xl font-bold mt-2">
    ₱48,500
  </h3>

</div>

<!-- Card 4 -->
<div class="bg-white p-6 rounded-xl shadow">

  <p class="text-gray-500">
    Customers
  <p>

  <h3 class="text-3xl font-bold mt-2">

```

86

</h3>

</div>

</div>

<!--=====RECENT ORDERS=====-->

<div class="bg-white rounded-xl shadow mt-8 p-6">

<h3 class="text-xl font-bold">

Recent Orders

</h3>

<div class="mt-6">

<div class="flex justify-between
border-b py-4">

ORD-001

Alex Student

¥1,590

Completed

</div>

<div class="flex justify-between
border-b py-4">

ORD-002

John Student

¥2,990

Pending

</div>

<div class="flex justify-between

```

        py-4">
        <span>ORD-003</span>

        <span>Jane Student</span>

        <span>฿890</span>

        <span class="text-blue-600">
            Shipping
        </span>

    </div>

</div>

</div>

<main>

</div>

```

5.5 วิเคราะห์โครงสร้าง

ตอนนี้เรามี:

```

<div class="min-h-screen bg-gray-100">
  |
  |----- Header
  |
  |----- Main
  |

```

Header

```

<header class="bg-white shadow-md">

```

สร้างพื้นหลังสีขาวและเงา

Container

```

<div class="max-w-7xl mx-auto px-6 py-4">

```

ทำหน้าที่

max-w-7xl กำหนดความกว้างสูงสุด

mx-auto จัดกึ่งกลาง

px-6 padding ซ้าย-ขวา

py-4 padding บน-ล่าง

5.6 Dashboard Cards

จุดสำคัญที่สุดคือ

```
<div class="grid grid-cols-4 gap-6">
```

ให้อ่านจากซ้ายไปขวา

grid -> ใช้ Grid

grid-cols-4 -> แบ่งเป็น 4 columns

gap-6 -> ระยะห่างระหว่าง Card

จึงได้

Products	Orders	Revenue	Customers
24	128	\$48,500	86

5.7 Recent Orders

ตรงนี้เราใช้

```
<div class="flex justify-between">
```

เพื่อจัด

โดย flex -> เรียงแนวนอน และ justify-between -> กระจายข้อมูล

นี่เป็นตัวอย่างที่ดีมากในการเปรียบเทียบ

Grid

ใช้เมื่อเราต้องการสร้าง โครงสร้างหลาย Columns

```
grid grid-cols-4
```

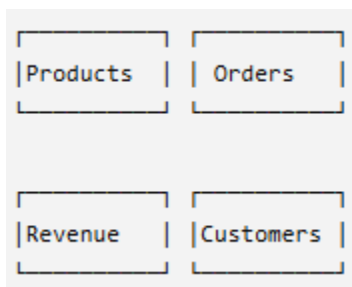
Flex

ใช้เมื่อเราต้องการ จัดตำแหน่งสมาชิกในหนึ่งแถว

```
flex justify-between
```

5.8 ทดลองแก้ไข Layout

ลองเปลี่ยน grid-cols-4 เป็น grid-cols-2 จะได้

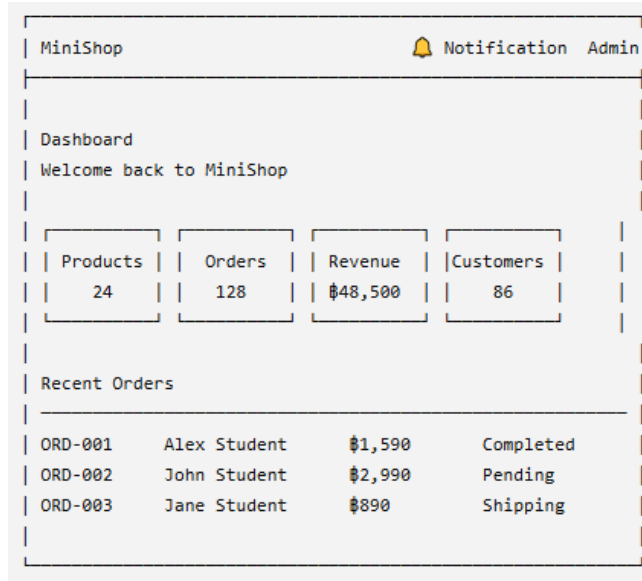


จากนั้นเปลี่ยนกลับ grid-cols-4

5.9 ทดลอง gap

เปลี่ยน gap-6 เป็น gap-2 จะเห็นว่า Card ชิดกันมากขึ้น จากนั้นลอง gap-10 จะเห็นช่องว่างมากขึ้น

เมื่อจบ Part 5 นักศึกษาควรจะได้ผลลัพธ์ดังนี้

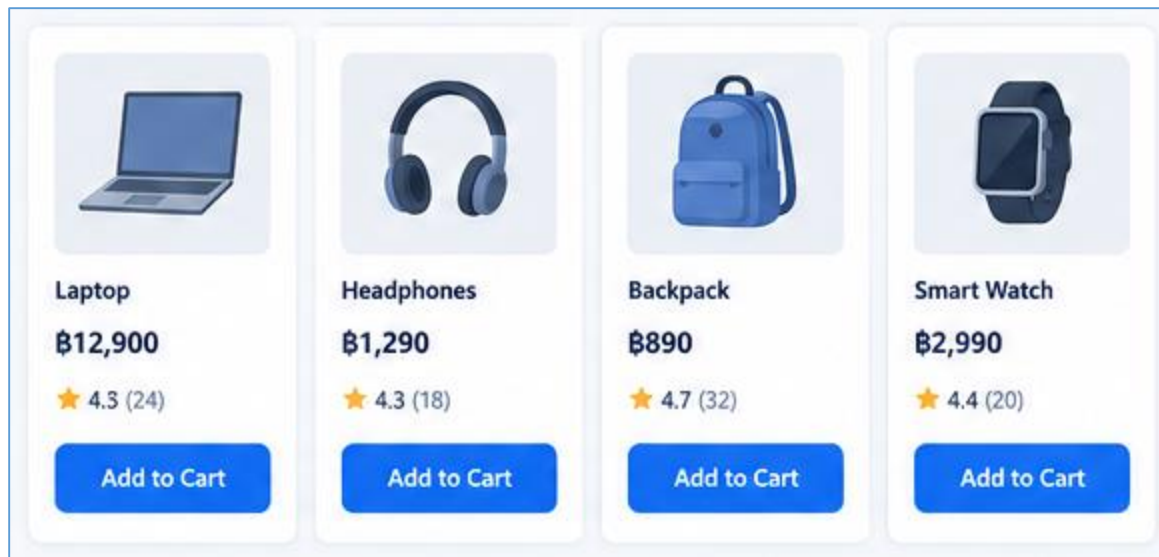


PART 6 — Product Cards

หลังจากนักศึกษาเข้าใจ Dashboard แล้ว เราจะนำ Grid มาใช้กับ Product

6.1 เป้าหมาย

สร้าง Product Cards:



6.2 Code

เพิ่มส่วนนี้ใน main.js

```
<div class="grid grid-cols-4 gap-6 mt-8">

  <div class="bg-white rounded-xl shadow p-5">

    <div class="h-40 bg-gray-100 rounded-lg
              flex items-center justify-center
              text-5xl">

      📁

    </div>

    <h3 class="text-lg font-bold mt-4">
      Laptop
    </h3>

    <p class="text-blue-600 font-bold mt-2">
      ฿12,900
    </p>
```

```

        <p class="text-yellow-500 mt-2">
            ★★★★★
        </p>

        <button
            class="w-full mt-4
                    bg-blue-600
                    text-white
                    py-2
                    rounded-lg
                    hover:bg-blue-700">
            Add to Cart

        </button>

    </div>

</div>

```

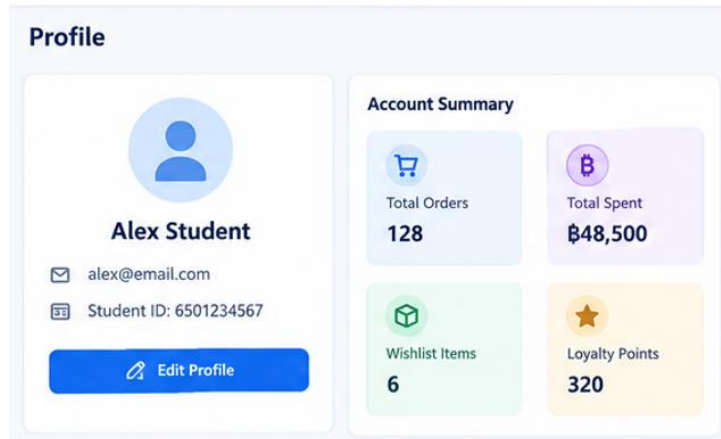
ให้นักศึกษาสร้างเพิ่มอีก 3 Card

- ☐ Headphones
- ☐ Backpack
- ☐ Smart Watch

PART 7 — Profile

สร้างหน้า Profile โดยใช้ Flexbox

โครงสร้าง



ตัวอย่าง

```
<div class="bg-white rounded-xl shadow p-8">

  <div class="flex items-center gap-6">

    <div class="w-24 h-24
      bg-blue-100
      rounded-full
      flex items-center justify-center
      text-4xl">

      <img alt="User profile icon" data-bbox="208 531 221 544"/>

    </div>

    <div>

      <h2 class="text-2xl font-bold">
        Alex Student
      </h2>

      <p class="text-gray-500">
        alex@example.com
      </p>

      <p class="text-gray-500">
        Student ID: 66123456
      </p>

    </div>

  </div>

</div>
```

PART 8 — Responsive Design

ตรงนี้เป็น Part ที่ สำคัญมาก เพราะ Tailwind จุดเด่นหนึ่งคือ Responsive Utility

แทนที่จะเขียน CSS Media Query เอง เราสามารถเขียน

```
grid-cols-1  
md:grid-cols-2  
lg:grid-cols-4
```

หมายถึง

หน้าจอเล็ก



1 column

หน้าจอขนาดกลาง



2 columns

หน้าจอขนาดใหญ่



4 columns

ดังนั้น Product

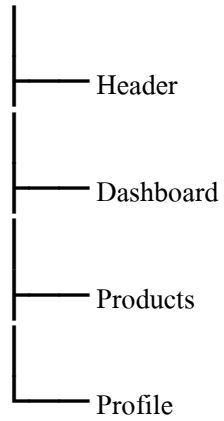
```
<div class="grid  
    grid-cols-1  
    md:grid-cols-2  
    lg:grid-cols-4  
    gap-6">
```

จะปรับตามขนาดหน้าจอ Mobile, Tablet, Desktop ให้สังเกตจำนวน Card ที่แสดงตามขนาดหน้าจอที่เปลี่ยนไป

PART 9 — MiniShop Integration

ตอนนี้ให้นักศึกษารวมทุกสิ่งที่เรียนมาให้เป็นโปรเจก

MiniShop



หน้าหลัก

